SectraTEK

AI FOR CYBERSECURITY AND DIGITAL FORENSICS · LAB 07

# Ask the Device



A small agent reads an entire Android extraction for you: ask a plain-English question and get an answer grounded in the image, with the exact file it used.

Daniel Kwaku Ntiamoah Addai · Mobile Threat Research Lead, SectraTEK Lab · AI for Cybersecurity and Digital Forensics

YOUR INSTRUCTOR

# Daniel Kwaku Ntiamoah Addai



Mobile Threat Research Lead

SectraTEK Lab

lab.sectratek.com

IN ONE LINE

I study how phones get spied on and tampered with, and build AI tools that help examiners find the evidence.

**Mobile threat research**
Spyware, stalkerware and device tampering.

**AI for investigators**
Machine learning that speeds up the search.

**Teaching**
AI for Cybersecurity and Digital Forensics.

TODAY'S STORY

# From a whole phone image to one cited answer

1

## Background

One answer hides in thousands of files.

2

## Before AI

Find the artifact, then hand-write SQL.

3

## The problem

Expert-bound and slow, or the model guesses.

4

## The gap

A chatbot knows Android, not your phone.

5

## How agents help

Give the model tools, and let it loop.

6

## The method

List, read, query: think, act, observe.

7

## The case

Which accounts are on this Pixel? The agent finds it.

8

## Your lab

Build the agent by hand in the browser.

Practical runs in Google Colab with a free hosted model. Nothing is installed on your computer.

BACKGROUND

# The examiner's needle in a haystack

A full phone extraction is thousands of files and dozens of databases. The answer to a simple question sits in exactly one of them, and you have to know where.

### A whole filesystem

App data, system files, media and logs: gigabytes per device.

### Dozens of databases

Accounts, contacts, SMS, chat apps: each its own SQLite schema.

### One artifact matters

“Which accounts are on this phone?” is answered by a single account record.

BEFORE AI

# The old way: find the file, then write SQL

## 1. Identify the location

You have to know that the Google account lives in a specific database, deep in the app-data tree, among thousands of other paths.

```
data/data/com.google.android.gm/
    databases/accounts.db
```

## 2. Write the SQL

Then you open that database and write a query by hand to pull the record you want out of the right table.

```
SELECT name, type
FROM accounts;
```

**THIS IS EXACTLY HOW IT HAS BEEN TAUGHT**

“I used to teach students to identify the correct location and use SQL to retrieve the answer.” It works, and it is precise, but it needs an expert who already knows the map of the device.

THE PROBLEM

# The shortcut, asking a model directly, invents the answer

ASK A MODEL: WHICH ACCOUNTS ARE ON THIS PHONE?

“Based on a typical Android device, most likely it is john.doe@gmail.com.”

NOTHING BEHIND IT

### It never saw this device

The model knows Android in general, not the contents of your extraction.

### So it guesses

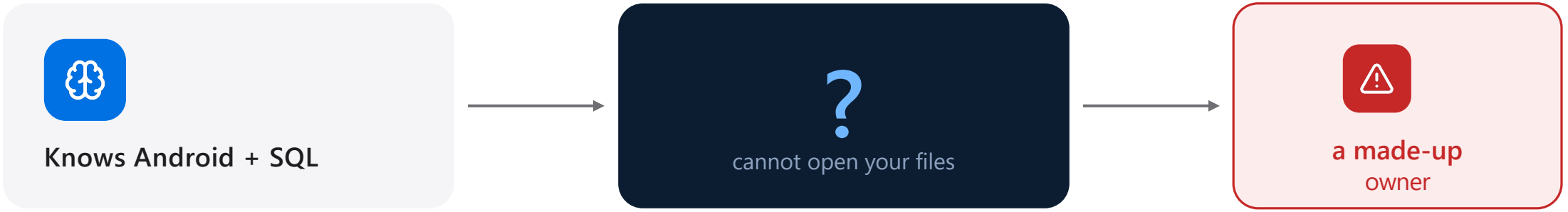
Asked which accounts are on the phone, it offers a fluent, plausible, fictional name.

### No file to check

There is no artifact behind the answer, nothing you can verify or cite.

THE GAP

# Knowledge, but no hands

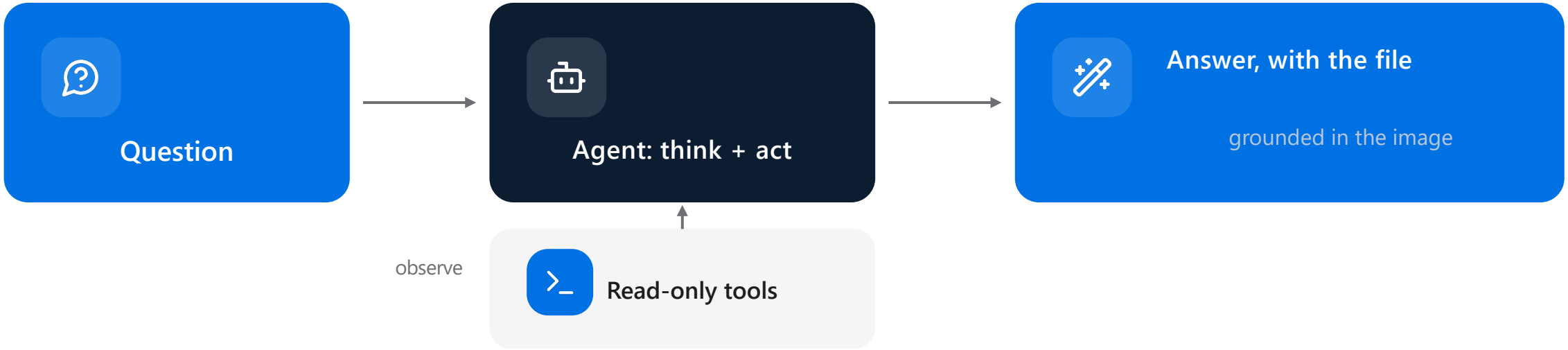



**GIVE THE MODEL HANDS**

The fix is not a bigger model. It is to give the model tools so it can open the evidence itself, look at what it finds, and answer only from that. A model that can act on the device, not just talk about it, is an agent.

THE ANSWER

# Think, act, observe, repeat




**IT ONLY SEES WHAT THE TOOLS RETURN**

It reads the evidence with tools instead of answering from memory, and the code checks it cites a file it opened.

**AND IT CITES THE FILE**

Every answer names the exact artifact it came from, so you can open it and check.

HOW IT WORKS

# Four moves, built by hand

### Think

Read the question and the last result, and decide the next move.

### Act

Call one read-only tool: list a folder, read a file, or run SQL.

### Observe

Take the tool's output as a new fact, and reason from it.

### Answer

When the evidence is enough, answer and cite the file.

**BEFORE AN ANSWER IS ACCEPTED: THE EVIDENCE CHECK**

The code, not the AI, checks that the answer cites a file that appears in the tool record. If not, the answer is sent back to open that file first. After two send-backs it is delivered marked UNVERIFIED.

No framework hides the agent: you write the tools, the loop, and the check yourself, in plain Python.

THE GUARDRAIL

# Five tools, and a hard boundary

The agent can do only five things, and all five are read-only and locked inside the image folder.

| | |
|---|---|
| list_dir(path) | See what is inside a folder. |
| read_file(path) | Read the text of a file. |
| run_sql(path, query) | Run one read-only SELECT on a database. |
| find_files(name) | Find files anywhere in the image by name. |
| search_text(keyword) | Search inside text files for a keyword. |

**WHY A BOUNDARY**

Read-only, and sealed inside the image, so the agent cannot change evidence or wander off the device, the way a loosely-scoped agent drifts into fetching a web page it was never meant to touch.

The same discipline as handling the real exhibit: look, never alter, and stay within the evidence.

THE PRACTICAL

# One question, the whole image

**WHAT'S IN THE IMAGE**

- Gmail / Google account
- Contacts
- SMS messages
- WhatsApp messages
- Installed apps
- Saved Wi-Fi networks
- Device properties

**WHAT THE AGENT DOES**

### Walk the image

Lists folders on its own until it finds the account database.

### Query the artifact

Writes the SQL to read the Google account record.

### Answer with a path

Names the owner and the exact file the answer came from.

SEE THE DIFFERENCE

# The same question, two ways

## Model alone

no device

*"Most likely john.doe@gmail.com."*

Fluent, invented, and no file to check.

## With the agent

read + cited

*"The signed-in accounts are nadia.okafor@gmail.com and a WhatsApp number."*

From the real account database, and it cites accounts.db.

PRACTICAL

# Your turn, in the browser

| | | |
|---|---|---|
| 1 | **Part 0: Set up** | 0.1 install, 0.2 connect to Groq (same free key as Labs 01-06), 0.3 build the practice phone image. |
| 2 | **Part A: Build the agent** | A.1 the five read-only tools, A.2 the rulebook, A.3 the loop with its evidence check. |
| 3 | **Part B: Why the agent matters** | B.1 ask the model with no tools, B.2 ask the same question through the agent. |
| 4 | **Part C: Your own questions** | C.1 ask about the phone itself, C.2 try your own and check each cited file. |
| 5 | **Write up and submit** | Paste an answer with its evidence path, compare B.1 and B.2, save the notebook. |

Every step explains its Purpose, Input, How it works, and Output. No key? Use 04_lab_notebook_local.ipynb.

TAKE IT FURTHER

# Point Sectra at a real extraction

The lab image is synthetic so it runs instantly. Sectra walks whatever tree you hand it, so you can aim it at real evidence three ways:

| | |
|---|---|
| **Synthetic (default)** | Built in one cell. Instant and identical for everyone: what the lab uses. |
| **A real public image** | Step R.2: a Pixel 3 / Android 13 full file system image from Digital Corpora (no login). |
| **Bring your own** | Already have an extraction folder? Call use_extraction('/path') and ask away. |

**REAL FULL-FILE-SYSTEM IMAGES ARE BIG: THAT IS HONEST, NOT A LIMITATION**

The public Pixel 3 image is about 18.5 GB to download and ~40 GB extracted: fine on a laptop or a roomy runtime. The fetch cell is left commented out, so nothing downloads until you choose, and the synthetic image stays the default.

SectraTEK

# Let the device answer for itself.

A forensic agent turns a plain-English question into a guided search of the evidence, and shows you the exact file behind every answer.

